WPT Day 8 Evening

Abhishek Purohit · Wed 28 Oct 26 · Client → API → database lifecycle, MySQL `productDB`, connection pools vs per-request connections, `mysql2/promise`, parameterized queries (SQL injection), and CRUD product APIs

---

Table of Contents

1. Why APIs Must Talk to a Database
2. Client-API-DB Lifecycle: Insert
3. Client-API-DB Lifecycle: Fetch
4. CRUD Naming and the Non-Negotiable Rule
5. MySQL Setup: productDB and the product Table
6. mysql2: Why a Library Is Required
7. Two Connection Approaches: Per-Request vs Pool
8. Connection Pool: Mechanism and Limits
9. mysql2 vs mysql2/promise and createPool
10. INSERT API: Body, Middleware, Try-Catch
11. SQL Injection and Parameterized Queries
12. Reading insertId from the Query Result
13. GET All Products and Why Fetch-All Is Dangerous
14. GET Product by ID and 404 Handling
15. DELETE by ID
16. Update Homework and Unique-Constraint Errors
17. Combined Worked Example: Full CRUD Server
18. Quick Revision Checklist
19. Practice Exercises
20. Practice Answers

Why APIs Must Talk to a Database

The problem APIs alone cannot solve

By Day 8 morning you already know how to create Express routes that behave as APIs. Those routes can receive JSON, compute something, and send a response. What they **cannot** do yet is **remember** anything after the process restarts.

Without a database:

- A registration form's data dies when the server restarts.
- A product catalogue cannot be shared across clients.
- There is no permanent home for insert, update, or delete.

What this session adds

The Express API (or any JavaScript server) must now **connect to an underlying database** — MySQL, MongoDB, Postgres, or another — and perform the fundamental operations: **insert, fetch, update, delete**.

The lecture's framing:

> We have to connect our APIs / Express application with the underlying database, and then the Express application can insert, fetch, update, or delete data.

Technology names change (Express vs Spring Boot vs Django). The **core principle stays the same**: client never talks to DB; API creates the connection and runs the query.

> **Exam:** Real-time apps need DB connectivity so the API can persist and retrieve data. Client never talks to the DB directly.

Client-API-DB Lifecycle: Insert

Architecture sketch

Three actors sit in a line. The server side holds **two** components:

```
[ Client ]                    [ Server side ]
web / mobile  ———request--->  [ Express API ]
(never talks                  [     |       ]
 to DB)                       [     v       ]
                              [  Database   ]
```

- **Client** — web app, iOS, Android. Sees a form; fills details; hits Submit.
- **API** — Express (in this course), or Java / Python / any backend.
- **Database** — permanent storage (MySQL here: `productDB`).

Why permanent storage

Data is saved in the DB **so it remains permanently** — registration / sign-up must survive process restarts and be available to later logins and other clients.

Insert flow step by step

1. Client fills a registration (or product) form and clicks Submit.
2. Frontend calls the registration / insert **API** (not the database).

3. Client sends a request whose **JSON request body** carries the data to save (morning session: request body).
4. API reads `request.body`.
5. API **creates a connection** with the database and fires an **INSERT** query built from that data.
6. Database returns success, a syntax/SQL error, or another failure result to the API.
7. API sends a **JSON response** back to the client — success or failure.

> Lecturer phrasing: *For the client, the database does not even exist. The client only calls the API and passes data in the request body.*

Exam-ready takeaway

Step	Who	What
1	Client	POST + JSON body
2	API	Read body, connect, INSERT
3	DB	Execute; return result
4	API	JSON response to client

> **Pitfall:** Imagining the frontend "runs INSERT in MySQL". Users never get DB access. Frontend → API → DB only.

Client-API-DB Lifecycle: Fetch

Same three actors, different query

Client still never opens the database. It asks the API for data that already lives in tables (for example products saved earlier).

Fetch flow step by step

1. Client needs product (or other) data that currently sits in the DB.
2. Client sends a request to the **API** only.
3. API creates a connection with the DB and fires a **SELECT** query.
4. DB runs the query and returns the result set to the API.

5. API converts table rows into **JSON** (typically an **array of JSON objects**) and puts that in the **response**.
6. Client receives JSON — never raw table rows over a direct DB wire.

DB shape vs API shape

Layer	Shape
Inside MySQL	Tables: rows and columns
On the wire to the client	JSON / array of JSON objects

The API is the translator. Same lifecycle applies to **UPDATE** and **DELETE**: client → API → connect → run query → respond.

> **Exam:** Fetch = client requests API → API SELECT → DB returns rows → API JSON response. Never client → DB.

CRUD Naming and the Non-Negotiable Rule

What CRUD means

The lecture names the four operations **CRUD**:

Letter	Meaning	Typical SQL
C	Create	`INSERT`
R	Read	`SELECT`
U	Update	`UPDATE`
D	Delete	`DELETE`

You have already done these **inside MySQL** by typing queries. In a real application you do **not** hand users the database and say "run INSERT to sign up".

The rule that never changes

1. Frontend calls the **API**.
2. API creates a **connection** with the database.

3. API runs the query and responds.

This holds for Java, Python, .NET, PHP, Express — **implementation differs; core principle stays the same**. Later Spring Boot + Hibernate will do the same job with different tools.

> **Exam:** CRUD = Create, Read, Update, Delete. Users get a frontend, not raw SQL access.

MySQL Setup: productDB and the product Table

Create the database

In MySQL 8 (lecture demo password: `root`):

```
CREATE DATABASE productDB;
USE productDB;
```

Create the `product` table

```
CREATE TABLE product (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(200),
  price FLOAT,
  description VARCHAR(300),
  quantity INT
);
```

(Lecture slip: a missing comma while typing — always check commas between column definitions.)

`DESCRIBE product` — what matters

Field	Notes from lecture
`id`	`INT`, NOT NULL, **PRIMARY KEY**, **AUTO_INCREMENT**
`name`	`VARCHAR(200)`
`price`	`FLOAT`
`description`	`VARCHAR(300)`
`quantity`	`INT`

Auto-increment consequence

On INSERT you send **name, price, description, quantity** — **not** `id`. MySQL assigns `id` starting from 1 automatically.

> **Exam:** Auto-increment PK means INSERT omits `id`; DB generates it.

---

mysql2: Why a Library Is Required

The problem

Express is one process; MySQL is another. You need a **communication layer** between them. Writing Windows/OS-level socket code yourself is not the course approach.

The solution: `mysql2`

mysql2 is a third-party Node module that supplies pre-defined functions to:

- create connections / pools
- send queries into MySQL
- get results back
- support insert, fetch, update, delete

It performs the low-level talk with the MySQL server so your Express code stays at the level of "pool + query".

Project bootstrap (lecture)

Folder: `CRUD APIs` under `28th October evening`.

```
npm init -y
```

Set `"type": "module"` in `package.json` for ECMAScript modules.

```
npm i express mysql2
```

One command installs both dependencies. Express for the HTTP API; mysql2 for DB connectivity.

> **Exam:** mysql2 simplifies connection + query execution. You install it; you do not invent the wire protocol.

> **Pitfall:** Installing mysql2 and never importing/using it does nothing. Install ≠ connect.

Two Connection Approaches: Per-Request vs Pool

Approach 1: new connection per request

Idea: every HTTP request opens a **brand-new** DB connection.

- 10 concurrent users → 10 new connections.
- Lakhs of users → lakhs of new connections.

Not feasible under real traffic. Server already under load then creates connections on every hit → application "explodes". Lecturer analogy: *code will crack, application will crack, then the manager will crack you*.

Acceptable only for **tiny** systems (lecture: "gully mithaiwala" scale). **Not** for enterprise / high traffic.

Approach 2: connection pool (the real-time rule)

For every real application — Express, Java, Python, .NET, PHP — **go with a connection pool**.

> **Exam:** Per-request connections = beginner / small systems only. Enterprise = **connection pool only**.

Connection Pool: Mechanism and Limits

Swimming-pool analogy (lecture)

A pool holds **ready-made connections** created when the server starts. The developer decides **how many** ready connections to keep.

Lifecycle of one request against the pool

1. Server starts → pool creates N ready connections (example later: 10).
2. A client request arrives → occupy **one** ready connection (no create-from-scratch wait).
3. Run the DB operation on that connection.
4. Response finishes → **release** the connection back into the pool (not destroy it).
5. Next clients reuse those same connections.

You are **not** constantly creating and closing connections. Connections stay alive; you **occupy** and **release**.

Example with limit 50

- Max 50 connections ready.
- One request occupies one → 49 free for others.
- Ten concurrent clients occupy ten → when done, all ten return to the pool.

When the limit is full: queue vs refuse

If all 50 are occupied, further clients may:

- **Wait in a queue** (you also choose a **queue limit**, e.g. 5), or
- Be **refused** immediately with a high-traffic / resources-unavailable style response.

If queue limit is **0**, no waiting — refuse when connection limit is exhausted. The choice depends on infrastructure and task nature.

```
Server start
     |
     v
[ Pool: ready connections 1..N ]
     |
request ⟶ occupy one ⟶ query ⟶ release back to pool
     |
if all N busy ⟶ queue (up to Q) or error / high traffic
```

> **Exam:** Pool = ready-made connections; occupy + release; set `connectionLimit` and optional `queueLimit`. Do not create a new connection per request in enterprise apps.

---

mysql2 vs mysql2/promise and createPool

Two import styles

Import	Behaviour
`mysql2`	Same function names; callback-style execution
`mysql2/promise`	Same functions; they **return Promises**

Lecture food analogy: plain vs "masala" — same dish family, promise variant returns a Promise you `await` (resolve or reject). Prefer **`mysql2/promise`**: more readable, avoids callback pyramids, more organized.

Beginners vs "legends"

- Beginners: `createConnection` (per-connection style).
- Enterprise / lecture path: **`createPool`**.

Calling `createPool` — configuration object

Pass an object with connection parameters:

Key	Lecture value	Meaning
`host`	`'localhost'`	Where MySQL runs (or full IP/URL in production)
`port`	`3306`	Default MySQL port unless changed
`user`	`'root'`	MySQL username
`password`	`'root'`	Lecture machine password — use yours
`database`	`'productDB'`	Which database to use
`connectionLimit`	`10`	Max ready connections in the pool
`queueLimit`	e.g. `0`	Max queued requests after limit; `0` = no queue

```
import { createPool } from "mysql2/promise";

const sqlConnectionPool = createPool({
  host: "localhost",
  port: 3306,
  user: "root",
  password: "root",
  database: "productDB",
  connectionLimit: 10,
  queueLimit: 0,
});
```

`createPool` returns a **pool object**. Keep that reference (`sqlConnectionPool`) for every later INSERT / SELECT / UPDATE / DELETE.

> **Exam:** Prefer `mysql2/promise` + `createPool`. Know host, port, user, password, database, connectionLimit, queueLimit.

INSERT API: Body, Middleware, Try-Catch

Minimal Express shell (lecture)

```
import express from "express";

const app = express();

app.get("/", (request, response) => {
  response.send({ message: "welcome to CRUD APIs" });
});

app.listen(7500);
```

Port **7500** in the demo — any free port is fine.

Route for create

- Method: `POST` (client sends body data).
- URL: `/product`.

Different operations get different routes (or same URL with different methods — see Read section).

Always wrap DB work in try / catch

DB work can fail (logic bugs, SQL errors, connection issues). If you do not catch:

- Client may **wait forever** with no response.

In `catch`:

- Log the error for debugging (`console.log(error)`).
- Send a client response — lecture starts with generic **"something went wrong"**.
- Status: use **500** for unknown server failures (default 200 is wrong for failures). Later, branch on error type for finer status codes.

```
app.post("/product", async (request, response) => {
  try {
    // insert logic
  } catch (error) {
```

```
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});
```

Request body contract (backend decides)

Omit `id`. Client sends:

```
{
  "name": "keyboard",
  "price": 754,
  "description": "wireless keyboard",
  "quantity": 10
}
```

Recall morning: `express.json()`

By default Express does **not** parse JSON bodies. Register:

```
app.use(express.json());
```

Then destructure:

```
const { name, price, description, quantity } = request.body;
```

Three steps of every DB write/read in this style

1. **Get data from the client** (body and/or URL params) — for INSERT, from body.
2. **Create the SQL query** (string) — with placeholders, not concatenated values.
3. **Run the query** via `sqlConnectionPool.query( ... )` and `await` the Promise (`async` on the handler).

> **Exam:** POST `/product` + `express.json()` + try/catch with 500 on unknown failure. Body keys: name, price, description, quantity.

SQL Injection and Parameterized Queries

The dangerous approach (demo, then reject)

Building SQL with template literals / concatenation:

```
const insertQuery = `INSERT INTO product (name, price, description, quantity)
VALUES ('${name}', ${price}, '${description}', ${quantity})`;
```

This is **100% prone to SQL injection**. Attacker-controlled strings can alter the SQL meaning. Lecture rule:

> Never ever try to concat your values inside of any query — insert, select, update, or delete.

Not Express-specific. Not mysql2-specific. **Any** stack can be injected if you concatenate.

Mentioned but out of syllabus for now

- **ORMs** (e.g. Sequelize, Prisma; Java: Hibernate) often help out of the box.
- Classic name: **prepared statements**.

Course approach: **parameterized queries with** `?` via mysql2 — same idea as prepared statements.

Safe pattern

1. Write SQL with `?` where values belong.
2. Pass values in a **separate array** as the second argument to `query`.
3. Driver sends query and values so replacement is not naive string glue.

```
const insertQuery = `INSERT INTO product (name, price, description, quantity)
VALUES (?, ?, ?, ?)`;

const [result] = await sqlConnectionPool.query(insertQuery, [
  name,
  price,
  description,
  quantity,
]);
```

Order of array values must match order of `?` placeholders.

How the two-step protection works (lecture intuition)

1. Query text with `?` goes toward the DB.
2. Values travel as bound parameters and replace placeholders safely.

Do **not** ship values already baked into the SQL string.

> **Exam:** Concatenating user data into SQL ⇒ SQL injection risk. Use `?` placeholders + values array. Same rule for SELECT/UPDATE/DELETE.

> **Pitfall:** Thinking backticks alone are "safe". Template literals that embed `${userInput}` are still concatenation.

Reading insertId from the Query Result

What mysql2 returns for INSERT

`query` resolves to an **array**. First element is a result object with fields such as:

- `affectedRows` — e.g. `1` after one insert
- `insertId` — auto-generated primary key (1, then 2, then 3, …)
- `fieldCount` — often `0` for INSERT

Prefer destructuring over `result[0]`

```
const [result] = await sqlConnectionPool.query(insertQuery, [
  name,
  price,
  description,
  quantity,
]);

console.log(result.insertId);

response.send({
  message: "product inserted",
  insertId: result.insertId,
});
```

Frontend may need the new id immediately — send it in the response.

Live demo data (lecture)

POST examples: keyboard, mouse, headphone — then `SELECT * FROM product` confirmed rows in MySQL.

> **Exam:** After INSERT, read `insertId` from the first element of the query result array (destructure `[result]`).

GET All Products and Why Fetch-All Is Dangerous

Same URL, different method

Method	URL	Role
POST	`/product`	Create
GET	`/product`	Read all

One URL, different handlers by **HTTP method** — completely valid.

Steps for read-all

1. No client body needed — return everything the query selects.
2. Create: `SELECT * FROM product` (no `?` if no parameters).
3. `await sqlConnectionPool.query(fetchAllProductsQuery)` — second argument omitted when there are no placeholders.

Shape of the SELECT result

`query` again returns an **array**:

- Index **0**: array of row objects (the data you want) — array of JSON-like objects.
- Index **1**: field / table descriptor metadata.

Destructure the data:

```
app.get("/product", async (request, response) => {
  try {
    const fetchAllProductsQuery = "SELECT * FROM product";
    const [result] = await sqlConnectionPool.query(fetchAllProductsQuery);
    response.send(result);
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});
```

Why enterprise apps must not ship "fetch all"

Learning demo: fine. Production: dangerous.

- Selecting **lakhs / 1 crore** rows without care can **crash or stall** the server.
- Network pipeline and browser choke on huge payloads.
- Indexing helps query cost but does not make "send the entire table" a good API design.

Pagination is the real replacement: clients request page numbers; API returns limited slices (explore as homework). Never send all records in one response in a real system.

> **Exam:** GET `/product` can share the path with POST. `query` result `[0]` is row data. Fetch-all is for learning; production uses pagination.

GET Product by ID and 404 Handling

Route shape

```
GET /product/:id
```

Example: `/product/1` → id `1`.

Steps

1. Read id: `const { id } = request.params;`
2. Query with placeholder: `SELECT * FROM product WHERE id = ?`
3. Run: `query(fetchByIdQuery, [id])`
4. Destructure `[result]` — still an **array of rows** (SELECT always yields a list shape).

Return a single object, not a one-element array

Because `id` is the **primary key**, at most one row matches. Frontend expects a **single object** for "get one", not `[ { ... } ]`.

```
if (result.length > 0) {
  response.send(result[0]);
} else {
  response.status(404).send({
    message: `product with ID ${id} not found`,
  });
}
```

Missing id (e.g. `/product/4` when only 1–3 exist) → **404** + clear message. Existing id → **200** + object.

> **Exam:** Parameterized `WHERE id = ?`. PK fetch → send `result[0]` or 404 if `length === 0`.

> **Pitfall:** Sending the raw array for a by-id fetch forces every frontend to unwrap `[0]` for no reason.

DELETE by ID

Route

```
DELETE /product/:id
```

Same path pattern as GET by id; **method** distinguishes fetch vs delete.

Steps

1. Read `id` from `request.params`.
2. `DELETE FROM product WHERE id = ?` — never concatenate id.
3. `await sqlConnectionPool.query(deleteQuery, [id])`.

Result object highlights `affectedRows` (e.g. `1` if a row was deleted; `insertId` stays 0).

Lecture verification: DELETE id `2`, then GET `/product/2` → 404 not found.

Homework check (lecture)

If nothing was deleted (`affectedRows === 0`), respond **404** — same spirit as fetch-by-id.

Search need not be only by PK: `WHERE name = ?`, `LIKE`, price ranges, etc. — whatever the query requires. PK is common, not mandatory for every lookup.

> **Exam:** DELETE `/product/:id` + `?` + values array. Confirm with affectedRows / follow-up GET.

Update Homework and Unique-Constraint Errors

Update API (assigned as try-it-yourself)

Same flow as INSERT; change the SQL and method:

Piece	Choice
Method	**PUT**
URL	`/product/:id`
Id	from **URL params**
New fields	from **request body**
SQL	`UPDATE ... WHERE id = ?` with placeholders for body fields + id

Unique column exercise

1. Add a **UNIQUE** constraint (e.g. on `name`, or another entity's phone).
2. Insert a duplicate → MySQL throws; naive catch → generic **500**.
3. Better: in `catch`, read `error.code` (MySQL error code) and respond with a clear message such as "this value has already been taken" / duplicate.

What is *not* in this session

- Linking frontend (React) yet — login, password hashing, TypeScript, ESLint, then React, **then** wire UI to these APIs.
- Doing the wiring in raw HTML was deliberately postponed ("Mahabharata").

> **Exam:** PUT `/product/:id` = params id + body data + UPDATE with `?`. Unique violations → inspect error code in catch, not only generic 500.

Combined Worked Example: Full CRUD Server

End-to-end file matching the lecture's patterns (including a sketched PUT). Requires MySQL with `productDB.product` as above, and `npm i express mysql2` with `"type": "module"`.

File: `index.js`

```
import express from "express";
import { createPool } from "mysql2/promise";

const app = express();
app.use(express.json());

const sqlConnectionPool = createPool({
  host: "localhost",
  port: 3306,
  user: "root",
  password: "root",
  database: "productDB",
  connectionLimit: 10,
  queueLimit: 0,
});

app.get("/", (request, response) => {
  response.send({ message: "welcome to CRUD APIs" });
});

// C - Create
app.post("/product", async (request, response) => {
  try {
    const { name, price, description, quantity } = request.body;
    const insertQuery = `INSERT INTO product (name, price, description, quantity)
VALUES (?, ?, ?, ?)`;
    const [result] = await sqlConnectionPool.query(insertQuery, [
      name,
      price,
      description,
      quantity,
    ]);
    response.send({
      message: "product inserted",
      insertId: result.insertId,
    });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

// R - Read all (learning only; prefer pagination in production)
app.get("/product", async (request, response) => {
  try {
    const fetchAllProductsQuery = "SELECT * FROM product";
    const [result] = await sqlConnectionPool.query(fetchAllProductsQuery);
    response.send(result);
  } catch (error) {
    console.log(error);
```

```
    response.status(500).send({ message: "something went wrong" });
  }
});

// R - Read by id
app.get("/product/:id", async (request, response) => {
  try {
    const { id } = request.params;
    const fetchByIdQuery = "SELECT * FROM product WHERE id = ?";
    const [result] = await sqlConnectionPool.query(fetchByIdQuery, [id]);
    if (result.length > 0) {
      response.send(result[0]);
    } else {
      response.status(404).send({
        message: `product with ID ${id} not found`,
      });
    }
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

// U - Update (homework pattern from lecture)
app.put("/product/:id", async (request, response) => {
  try {
    const { id } = request.params;
    const { name, price, description, quantity } = request.body;
    const updateQuery = `UPDATE product
SET name = ?, price = ?, description = ?, quantity = ?
WHERE id = ?`;
    const [result] = await sqlConnectionPool.query(updateQuery, [
      name,
      price,
      description,
      quantity,
      id,
    ]);
    if (result.affectedRows === 0) {
      response.status(404).send({
        message: `product with ID ${id} not found`,
      });
      return;
    }
    response.send({ message: "product updated" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});
```

```
// D - Delete
app.delete("/product/:id", async (request, response) => {
  try {
    const { id } = request.params;
    const deleteQuery = "DELETE FROM product WHERE id = ?";
    const [result] = await sqlConnectionPool.query(deleteQuery, [id]);
    if (result.affectedRows == 0) {
      response.status(404).send({
        message: `product with ID ${id} not found`,
      });
      return;
    }
    response.send({ message: "product deleted" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

app.listen(7500, () => {
  console.log("CRUD APIs listening on 7500");
});
```

End-to-end trace (insert then read)

1. `node index.js` → pool creates up to 10 ready connections to `productDB`; server listens on 7500.
2. Postman: **POST** `http://localhost:7500/product`, Body raw JSON:

```
{
  "name": "headphone",
  "price": 1387,
  "description": "wireless headphone with mic",
  "quantity": 16
}
```

3. `express.json()` fills `request.body` → destructure → parameterized INSERT → MySQL assigns next `id`.
4. Response: `{ "message": "product inserted", "insertId": 3 }` (example).
5. **GET** `http://localhost:7500/product/3` → SELECT with `?` → single object, status 200.
6. **DELETE** `http://localhost:7500/product/3` → `affectedRows` 1 → later GET returns 404.

Quick Revision Checklist

Topic	One-line takeaway
Why DB + API	APIs alone do not permanently store data; connect Express to MySQL (or other DB) for CRUD.
Insert lifecycle	Client → API (JSON body) → INSERT → DB result → JSON response; client never hits DB.
Fetch lifecycle	Client → API → SELECT → rows → API converts to JSON array/objects → response.
CRUD	Create / Read / Update / Delete — users get UI + APIs, not raw SQL access.
product table	`id` AUTO_INCREMENT PK; INSERT supplies name, price, description, quantity only.
mysql2	Third-party module for connections and queries; install with Express.
Per-request connection	New connection every request — only tiny apps; explodes under load.
Connection pool	Ready-made connections; occupy + release; set connectionLimit and queueLimit.
mysql2/promise	Same APIs returning Promises; prefer over callbacks; use `createPool`.
createPool config	host, port, user, password, database, connectionLimit, queueLimit.
express.json	Required or `request.body` stays unusable for JSON POSTs.
try/catch	Always; unknown failure → 500 + "something went wrong".
SQL injection	Never concatenate user values into SQL; use `?` + values array.
insertId	From first element of INSERT result; send to client if needed.
Same URL	POST vs GET `/product` distinguished by method.
Fetch-all	Learning only; production → pagination (server/browser would choke).
GET by id	`/product/:id`, return `result[0]` or 404 if empty.
DELETE	Same path pattern, DELETE method; check affectedRows for 404.
PUT update	Id in params, new data in body, parameterized UPDATE.
Unique errors	Catch MySQL error code; return "already taken", not only generic 500.

Practice Exercises

Concept

1. Why must the client never communicate with the database directly in the architecture drawn in this lecture?
2. Expand CRUD and map each letter to a typical SQL statement.
3. What is wrong with opening a brand-new MySQL connection for every HTTP request on a high-traffic API?
4. In the swimming-pool analogy, what happens to a connection after the client's response is finished?
5. State two differences between importing `mysql2` and `mysql2/promise`.
6. Why is embedding `${name}` inside an INSERT string considered unsafe even when you "trust" the frontend?

Application

7. Write the `CREATE TABLE` for `product` as used in the lecture (column names and types).
8. Write a `createPool` config object for local MySQL, database `productDB`, limit 10 connections, no queue.
9. Write the parameterized INSERT for the product columns (excluding `id`) and the matching `query` call values array order.
10. Given GET `/product/:id` returned `[{ "id": 1, "name": "keyboard", ... }]`, what should the API send to the client instead, and why?
11. Sketch PUT `/product/:id`: which data comes from params vs body, and show an `UPDATE` with placeholders.

Predict output

12. After three successful INSERTs into an empty auto-increment `product` table, what is `insertId` on the third insert's result object?
13. `SELECT *` via `pool.query` returns `[rows, fields]`. You write `const [result] = await pool.query( ... )` then `response.send(result)`. What does the client receive?
14. GET `/product/4` when no row has `id = 4`, and the handler checks `result.length` as in the lecture. Status code and message shape?

Spot the bug

15. Find the bugs:

```
app.post("/product", (request, response) => {
  const { name, price, description, quantity } = request.body;
  const q = `INSERT INTO product (name, price, description, quantity)
VALUES ('${name}', ${price}, '${description}', ${quantity})`;
  sqlConnectionPool.query(q);
  response.send({ message: "ok" });
});
```

16. Find the bugs:

```
app.get("/product/:id", async (request, response) => {
  try {
    const { id } = request.params;
    const [result] = await sqlConnectionPool.query(
      "SELECT * FROM product WHERE id = " + id
    );
    response.send(result);
  } catch (error) {
    response.send({ message: "something went wrong" });
  }
});
```

Practice Answers

1. The client only talks to the API. The API alone connects to the DB and runs queries. That keeps credentials, SQL, and schema off the client; every real-time app follows client → API → DB.

2. Create → INSERT; Read → SELECT; Update → UPDATE; Delete → DELETE.

3. Connection count grows with traffic (10 users → 10 connections; lakhs → lakhs). Under load the server spends itself creating connections and can crash. Only tiny apps use per-request connections.

4. It is **released / restored** into the pool (not permanently destroyed). Later requests can occupy it again.

5. Function names/arguments stay the same; `mysql2/promise` returns Promises (await-friendly, no callbacks), plain `mysql2` is callback-oriented. Lecture prefers promise variant for readability.

6. Any concatenated user-controlled string can change SQL meaning (SQL injection). Trusting the UI is irrelevant — attackers can call the API directly. Use `?` placeholders and a values array.

7.

```sql
CREATE TABLE product (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(200),
  price FLOAT,
  description VARCHAR(300),
  quantity INT
);
```

8.

```
{
  host: "localhost",
  port: 3306,
  user: "root",
  password: "root",
  database: "productDB",
  connectionLimit: 10,
  queueLimit: 0,
}
```

9.

```
const insertQuery = `INSERT INTO product (name, price, description, quantity)
VALUES (?, ?, ?, ?)`;
await sqlConnectionPool.query(insertQuery, [name, price, description, quantity]);
```

10. Send `result[0]` (the single object). Id is the primary key, so at most one row; returning an array forces the frontend to unwrap index 0 for no benefit.

11. `id` from `request.params`; `name`, `price`, `description`, `quantity` from `request.body`.

```
UPDATE product
SET name = ?, price = ?, description = ?, quantity = ?
WHERE id = ?
```

Values array: `[name, price, description, quantity, id]`.

12. `3` (auto-increment 1, then 2, then 3).

13. The array of row objects (all products as JSON-like objects) — the first element of mysql2's return tuple.

14. Status **404**; body like `{ message: "product with ID 4 not found" }`.

15.

- No `express.json()` visible — `request.body` may be useless.
- Handler not `async`; Promise from `query` not `await`ed.
- Values concatenated into SQL → **SQL injection**.
- No try/catch; failures may leave the client without a proper error response.
- Ignores query result / `insertId`.

16.

- Concatenates `id` into SQL → injection risk; should use `WHERE id = ?` and `[id]`.
- Sends full `result` array instead of `result[0]` for a by-id fetch.
- No empty-array → 404 branch.
- Catch uses default status 200 instead of **500**.